

JavaFX常用控件之



使用菜单

北京理工大学计算机学院
金旭亮

概述



菜单是非常常见的一种控件，在JavaFX应用中，它通常会与BorderPane相互配合，放置在Top区，显示于屏幕顶部。



菜单项是MenuItem的实例，它定义了菜单项显示的文本和图标，并且可以给它附加单击事件响应代码。

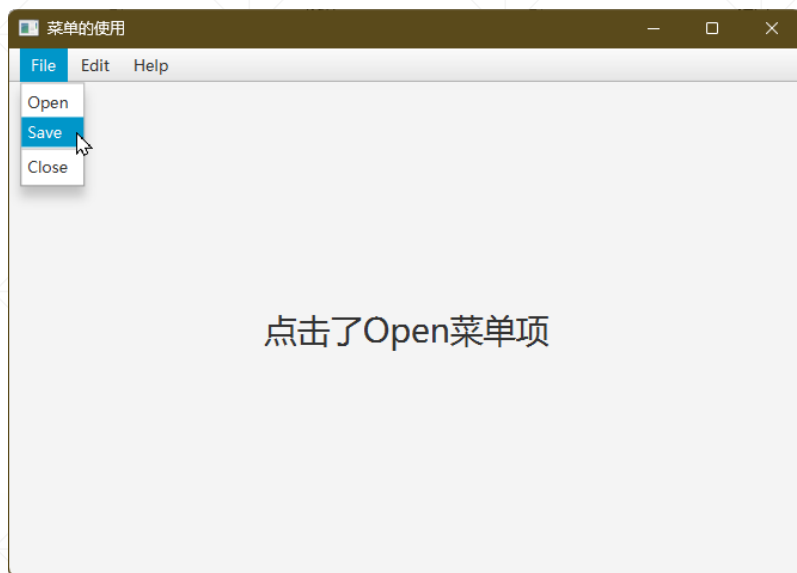


Menu是MenuItem的容器，用于显示多个菜单项。



MenuItem需要加入到Menu中，Menu再加入到MenuBar中，最后，再将MenuBar追加到场景图中，才能显示菜单。

在FXML中声明下拉菜单



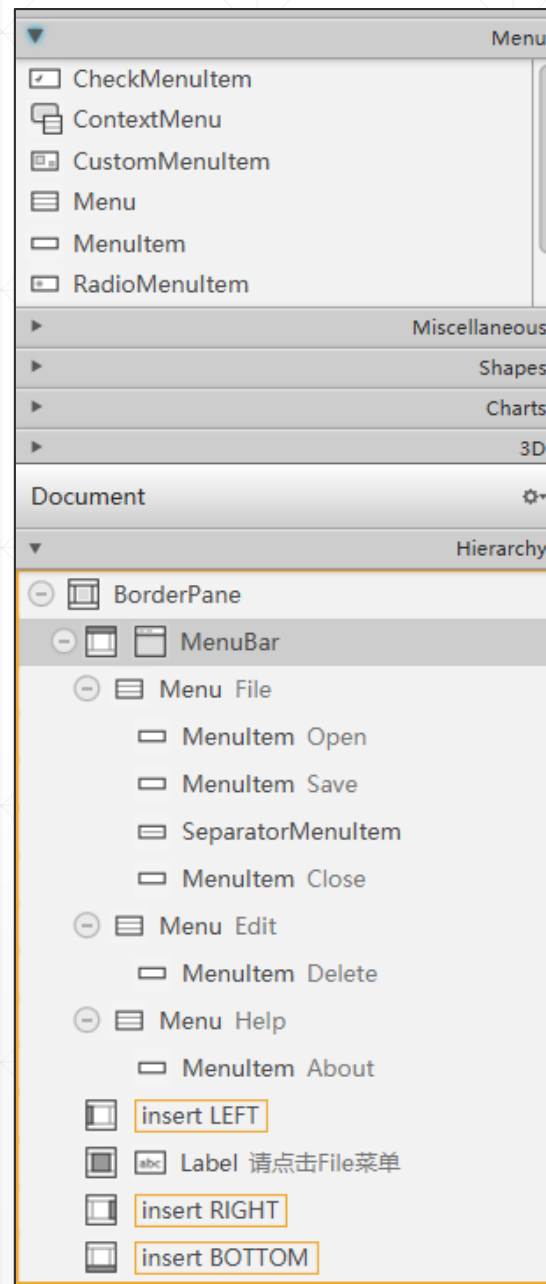
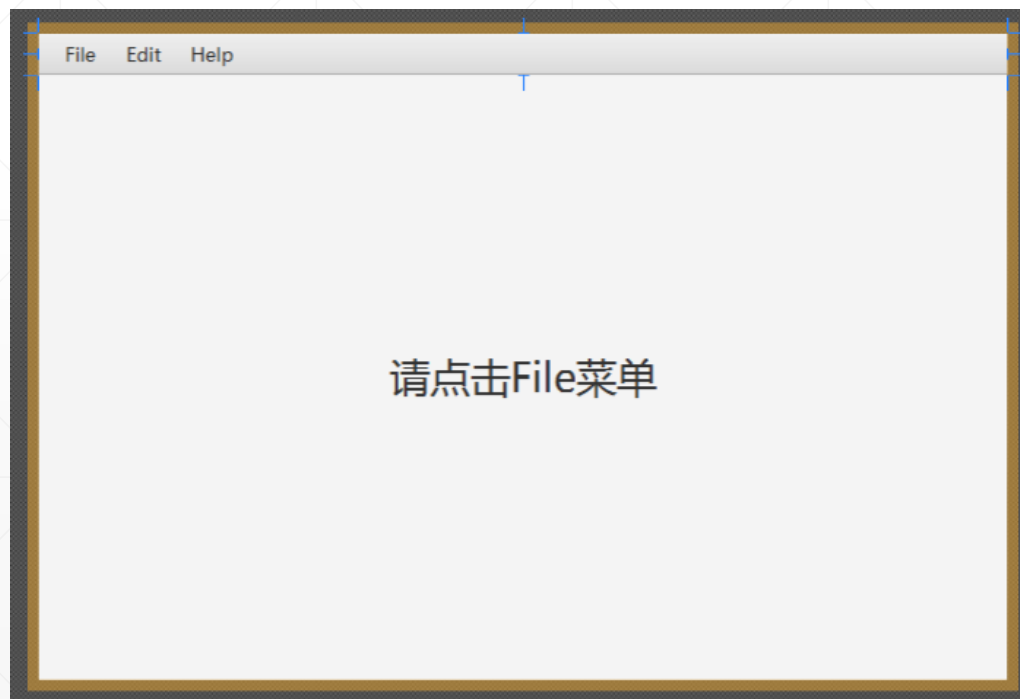
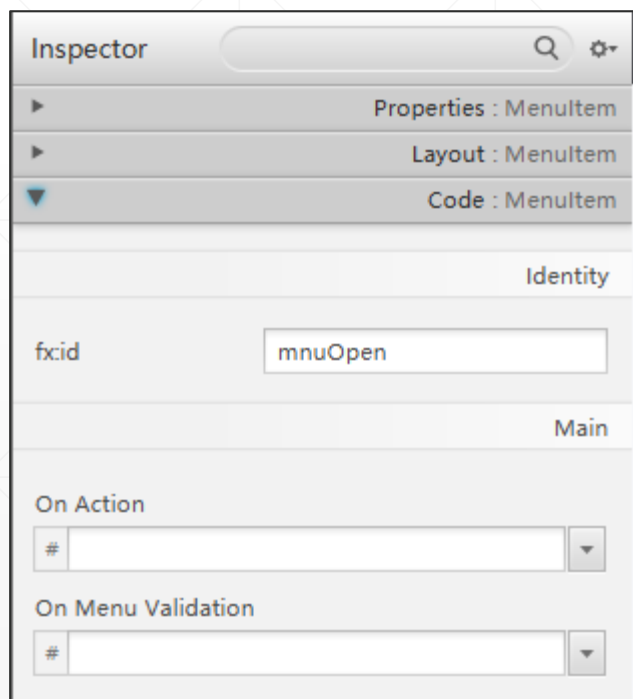
采用MVC架构的菜单示例

```
<MenuBar BorderPane.alignment="CENTER">
  <menus>
    <Menu mnemonicParsing="false" text="File">
      <items>
        <MenuItem fx:id="mnuOpen" mnemonicParsing="false" text="Open" />
        <MenuItem fx:id="mnuSave" mnemonicParsing="false" text="Save" />
        <SeparatorMenuItem mnemonicParsing="false" />
        <MenuItem fx:id="mnuClose" mnemonicParsing="false" text="Close" />
      </items>
    </Menu>
    <Menu mnemonicParsing="false" text="Edit" ...>
    <Menu mnemonicParsing="false" text="Help" ...>
  </menus>
</MenuBar>
```

在FXML中声明的菜单

菜单的可视化设计

在SceneBuilder中，可以直接使用Menu相关组件，以可视化的方式设计JavaFX应用的菜单，并设置相应的菜单组件属性。



控制器中引用菜单项对象

```
public class MenuController implements Initializable {  
    2 usages  
    @FXML  
    private MenuItem mnuOpen;  
    2 usages  
    @FXML  
    private MenuItem mnuSave;  
    2 usages  
    @FXML  
    private MenuItem mnuClose;  
    3 usages  
    @FXML  
    private Label lblInfo;  
}
```

引用三个
菜单项

由于在实际开发中，经常需要给特定的菜单项添加点击事件响应代码，因此，通常会在控制器中为这些菜单项声明相应的字段。

注意字段名，要与菜单项的id值一致。

控制器中为菜单挂接事件响应代码

让控制器实现Initializable接口，在initialize方法响应菜单事件。

```
@Override
public void initialize(URL url, ResourceBundle resourceBundle) {
    mnuClose.setOnAction(e->{
        Platform.exit();
    });

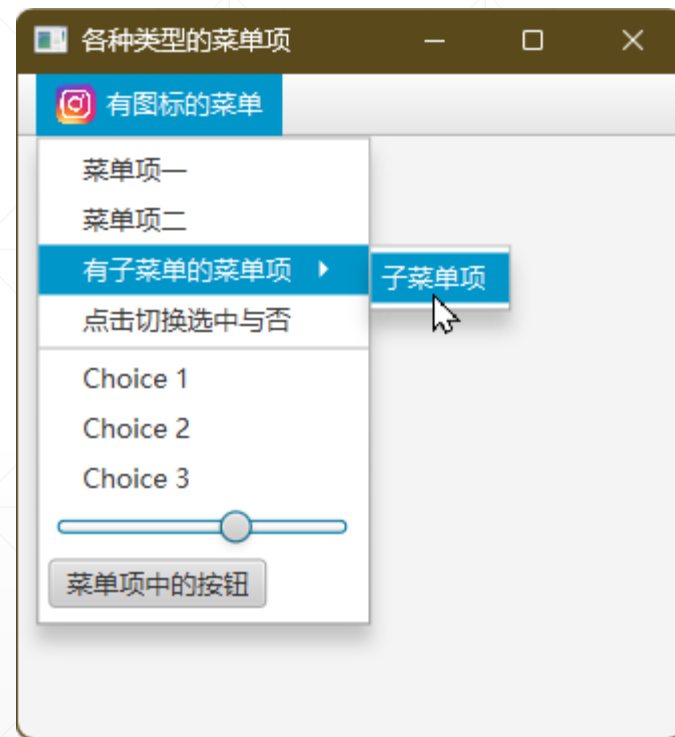
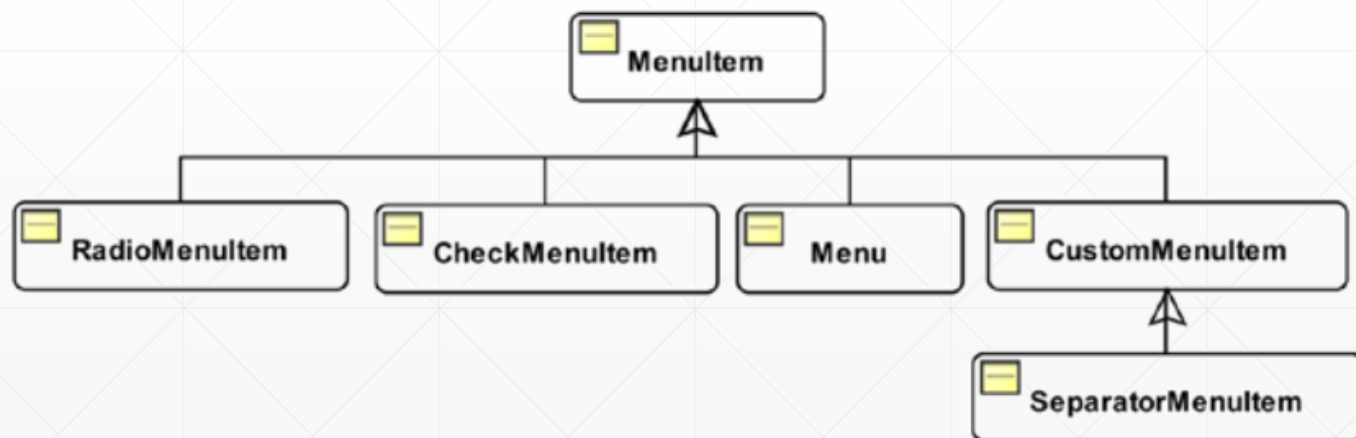
    mnuOpen.setOnAction(e->{
        lblInfo.setText("点击了Open菜单项");
    });

    mnuSave.setOnAction(e->{
        lblInfo.setText("点击了Save菜单项");
    });
}
```

为菜单项点击事件
挂接事件响应代码

多种类型的菜单项

最简单的菜单项就是显示一个文本，复杂一点的可以显示一个图标，或者附加上一些额外的交互特性，比如“互斥选中”或“仅作为分割条显示，不触发鼠标点击事件”等等，JavaFX提供了相应的MenuItem子类型完成这一工作。



UseMultiTypeMenuItem.java

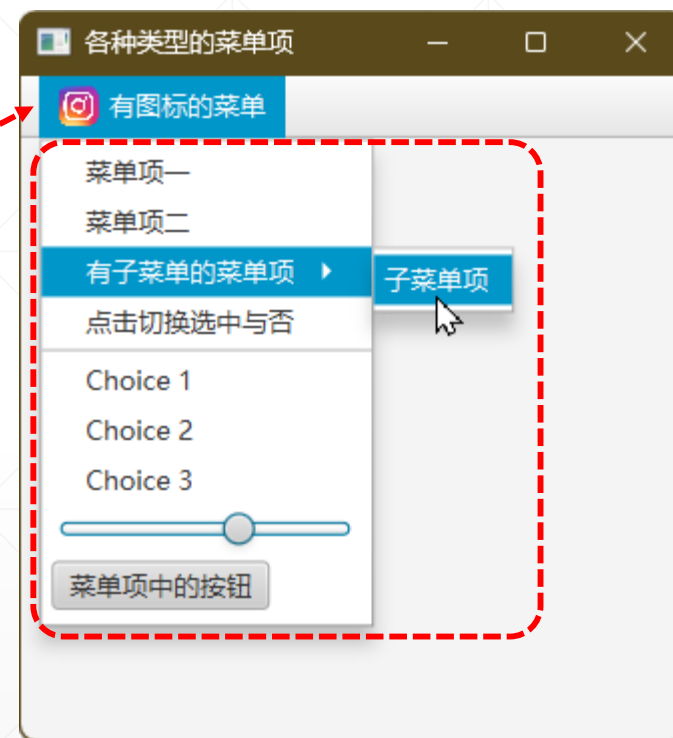
UseMultiTypeMenuItem示例分析-1

示例代码框架

```
//用于承载菜单的容器控件
MenuBar menuBar = new MenuBar();

Menu menu = new Menu("有图标的菜单");
//....(添加各种菜单项)
menuBar.getMenus().add(menu);

VBox vBox = new VBox(menuBar);
Scene scene = new Scene(vBox, 300, 300);
primaryStage.setScene(scene);
primaryStage.setTitle("各种类型的菜单项");
primaryStage.show();
```



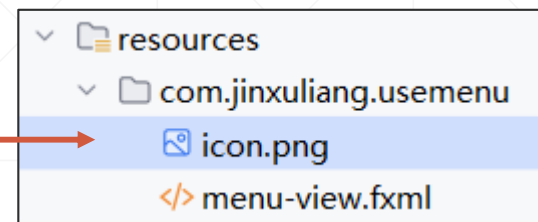
UseMultiTypeMenuItem示例分析-2

显示图标

```
Menu menu = new Menu( text: "有图标的菜单");  
//从资源中加载图标  
var imageUrl= MenuApplication.class.getResource( name: "icon.png");  
FileInputStream input = new FileInputStream(imageUrl.getPath());  
Image image = new Image(input);  
ImageView imageView = new ImageView(image);  
imageView.setFitHeight(20);  
imageView.setFitWidth(20);  
//将菜单与图标关联起来  
menu.setGraphic(imageView);
```

使用这个方法显示图标

设定图标的大小



图标放到资源文件夹中

图标文件所在的文件夹名与使用它的java类所在的包名一致，就能直接通过文件名找到这个图标文件，可以简化代码。

UseMultiTypeMenuItem示例分析-3

//普通菜单项及点击事件响应

```
MenuItem menuItem1 = new MenuItem( text: "菜单项一");  
menuItem1.setOnAction(e -> {  
    System.out.println("菜单项一被选中");  
});  
MenuItem menuItem2 = new MenuItem( text: "菜单项二");  
menu.getItems().add(menuItem1);  
menu.getItems().add(menuItem2);
```

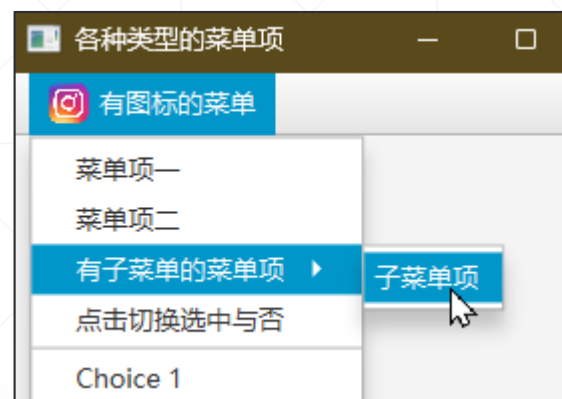


点击菜单项一，会在控制台窗口看到相应的输出。

UseMultiTypeMenuItem示例分析-4

可以通过MenuItem的items集合，实现多级子菜单：

```
Menu subMenu = new Menu( text: "有子菜单的菜单项");  
MenuItem menuItem11 = new MenuItem( text: "子菜单项");  
subMenu.getItems().add(menuItem11);  
//添加子菜单项  
menu.getItems().add(subMenu);
```



嵌套层数过多的菜单项，不易于使用，慎用。

UseMultiTypeMenuItem示例分析-5

//支持选中的菜单项

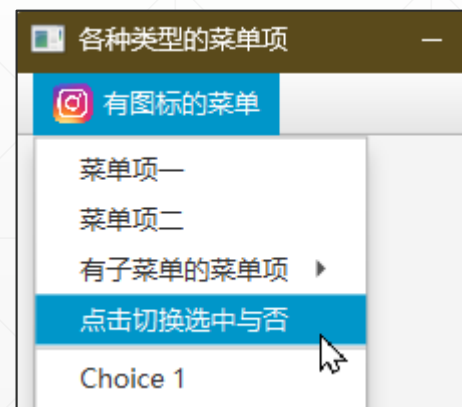
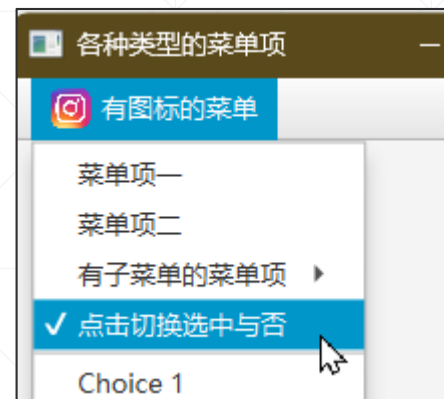
```
CheckMenuItem checkMenuItem = new CheckMenuItem( text: "点击切换选中与否");  
menu.getItems().add(checkMenuItem);
```

//菜单分割条

```
SeparatorMenuItem separator = new SeparatorMenuItem();  
menu.getItems().add(separator);
```



通常情况下，菜单分割条仅仅只用于显示，不响应点击事件。



UseMultiTypeMenuItem示例分析-6

将RadioMenuItem加入到ToggleGroup中，可以定义一个“一次只能选中一项”的菜单区域。

//归为一组的菜单项，互斥选中

```
RadioMenuItem choice1Item = new RadioMenuItem(text: "Choice 1");
```

```
RadioMenuItem choice2Item = new RadioMenuItem(text: "Choice 2");
```

```
RadioMenuItem choice3Item = new RadioMenuItem(text: "Choice 3");
```

```
ToggleGroup toggleGroup = new ToggleGroup();
```

```
toggleGroup.getToggles().add(choice1Item);
```

```
toggleGroup.getToggles().add(choice2Item);
```

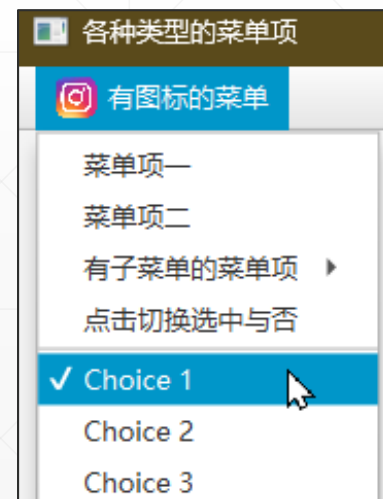
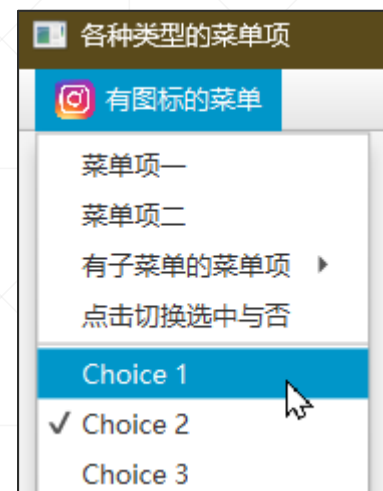
```
toggleGroup.getToggles().add(choice3Item);
```

//将上述菜单项加入到菜单组件中

```
menu.getItems().add(choice1Item);
```

```
menu.getItems().add(choice2Item);
```

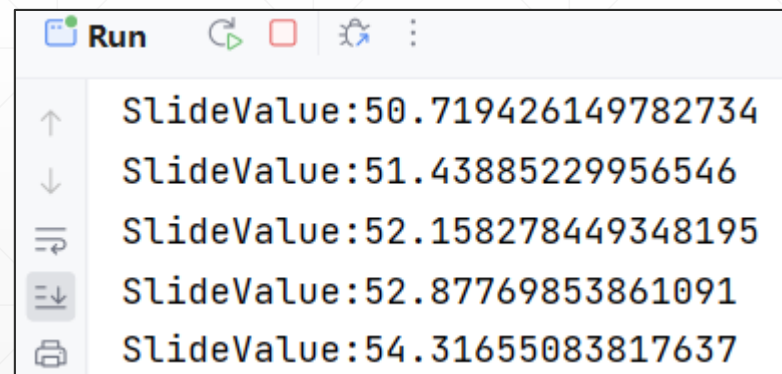
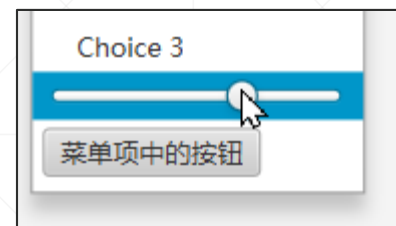
```
menu.getItems().add(choice3Item);
```



UseMultiTypeMenuItem示例分析-7

使用CustomMenuItem，可以自定义菜单显示的内容：

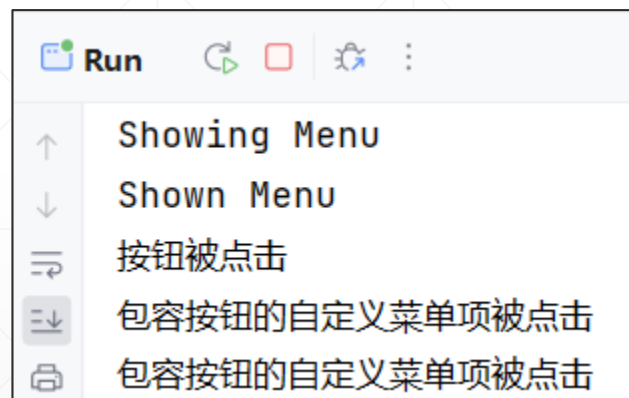
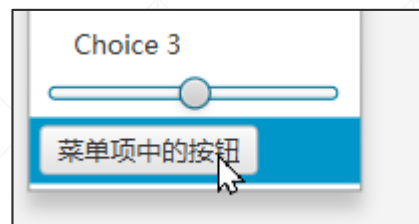
```
//创建一个Slider控件
Slider slider = new Slider( min: 0, max: 100, value: 50);
//响应滑块事件
slider.valueProperty().addListener(ctl->{
    System.out.println("SlideValue:"+slider.getValue());
});
//定义一个自定义菜单控件
CustomMenuItem customMenuItem = new CustomMenuItem();
//以滑块控件作为显示内容
customMenuItem.setContent(slider);
customMenuItem.setHideOnClick(false);
menu.getItems().add(customMenuItem);
```



UseMultiTypeMenuItem示例分析-8

//创建一个使用按钮作为显示内容的菜单项

```
Button button = new Button( text: "菜单项中的按钮");
button.setOnAction(e->{
    System.out.println("按钮被点击");
});
CustomMenuItem customMenuItem2 = new CustomMenuItem();
customMenuItem2.setContent(button);
customMenuItem2.setHideOnClick(false);
//注意: 以下事件响应代码将被调用两次
customMenuItem2.setOnAction(e->{
    System.out.println("包容按钮的自定义菜单项被点击");
});
menu.getItems().add(customMenuItem2);
```



将其它控件集成到菜单中，应该直接为这一控件的相应事件编写响应代码，而不要再设置菜单项本身的Click事件响应代码。

UseMultiTypeMenuItem示例分析-9

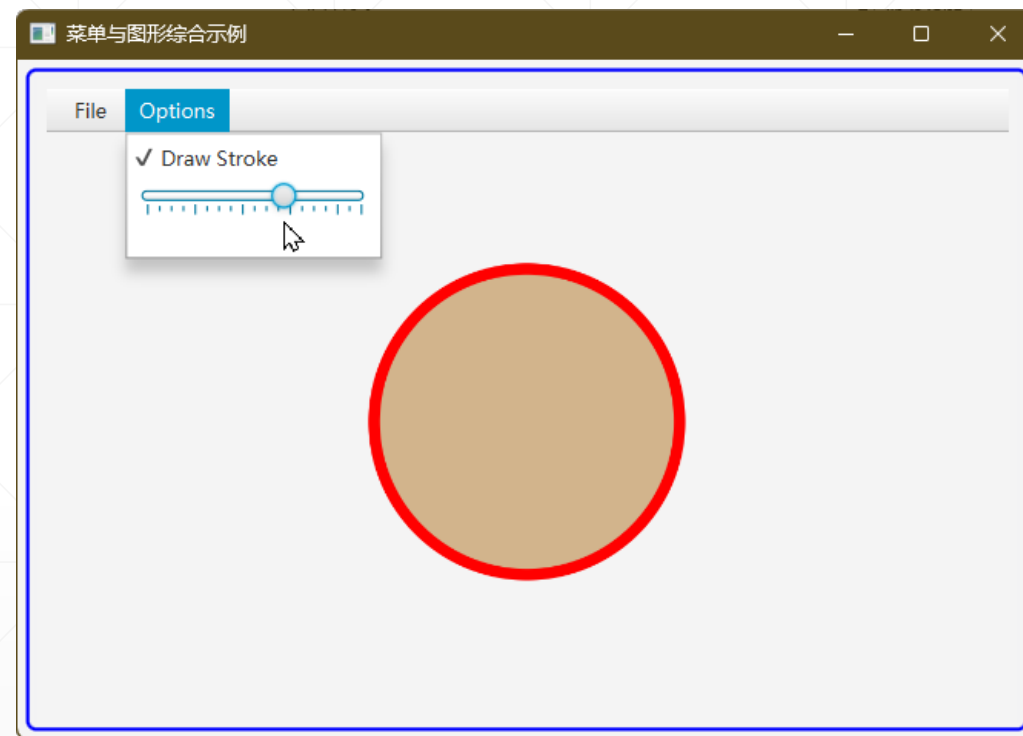
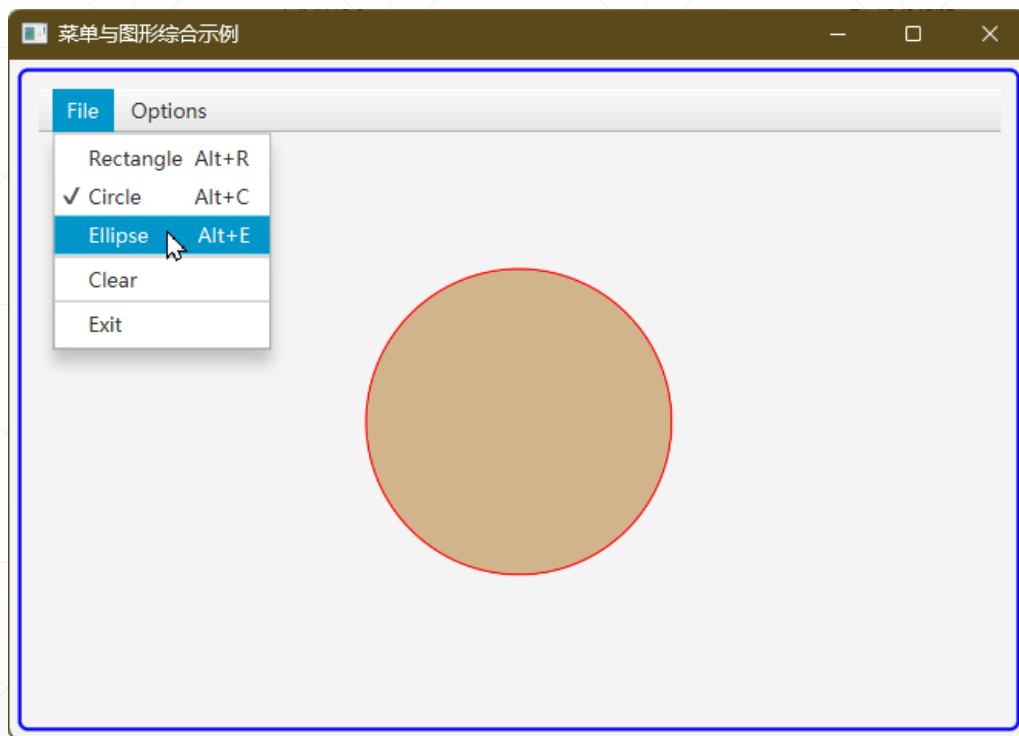


菜单的显示与隐藏，会触发相应的事件，如果需要的话，可以为其编写事件响应代码，如下所示：

```
//响应菜单项的“显示”与“隐藏”事件
```

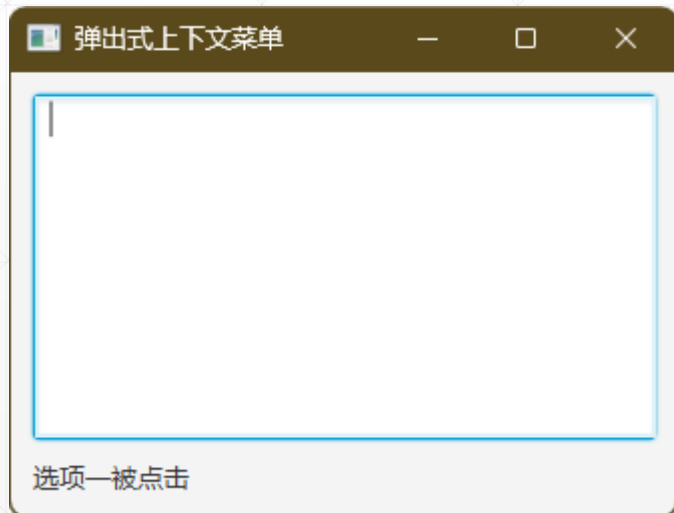
```
menu.setOnShowing(e -> { System.out.println("Showing Menu"); });  
menu.setOnShown (e -> { System.out.println("Shown Menu"); });  
menu.setOnHiding (e -> { System.out.println("Hiding Menu"); });  
menu.setOnHidden (e -> { System.out.println("Hidden Menu"); });
```

下拉菜单综合示例



MenuAndShape 示例，组合了菜单及图形功能，编写了一个简单的绘图程序，请课后自行阅读其示例源码。

弹出式上下文菜单



```
Label lblInfo = new Label( text: "右击文本编辑框，显示弹出菜单");
```

//构建弹出式菜单

```
ContextMenu contextMenu = new ContextMenu();
```

```
MenuItem menuItem1 = new MenuItem( text: "选项一");
```

```
MenuItem menuItem2 = new MenuItem( text: "选项二");
```

```
MenuItem menuItem3 = new MenuItem( text: "选项三");
```

```
menuItem1.setOnAction((event) -> {
```

```
    lblInfo.setText("选项一被点击");
```

```
});
```

```
contextMenu.getItems().addAll(menuItem1, menuItem2, menuItem3);
```

//使用自定义菜单替换掉TextArea内置的菜单

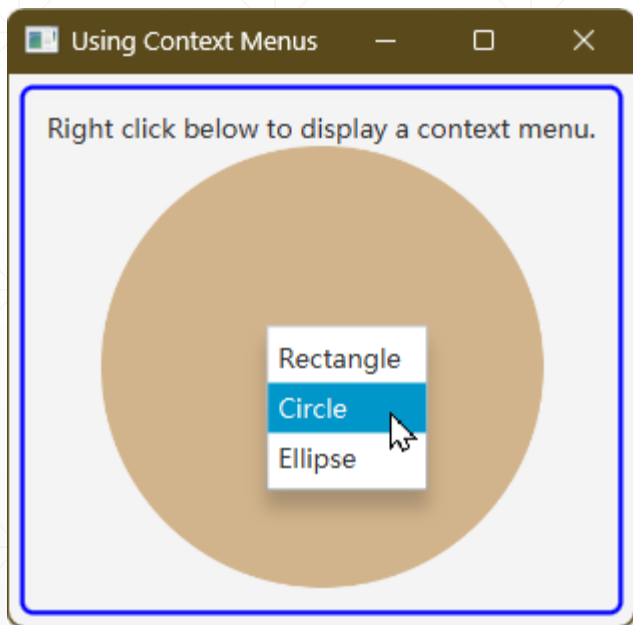
```
TextArea textArea = new TextArea();
```

```
textArea.setContextMenu(contextMenu);
```

练习



如何编写代码实现以下功能？请动手试一试！



参考答案：
`ContextMenuAndShape.java`